

RELATÓRIO ATIVIDADE 1 – ANÁLISE DE LLMS PARA PADRÕES ARQUITETURAIS

Projeto utilizado para análise: [MoneyPrinterTurbo](#)

Repositório com os artefatos produzidos: [GitHub](#)

Vídeo: [Drive](#)

Integrantes: Fernanda da Silva Santos, Gustavo Rodrigues dos Santos, João Santos Rocha, José Henrique Souza Santana, Jose Luiz de Jesus Souza, Lais Santos de Sousa, Rafael de Jesus Santos, Tasso Marcel de Oliveira.

Resumo básico do projeto: Geração automática de vídeos curtos a partir de um tópico, usando LLMs.

JUSTIFICATIVA DA ESCOLHA DO PROJETO

O projeto MoneyPrinterTurbo (harry0703/MoneyPrinterTurbo) foi selecionado para esta análise arquitetural por sua alta relevância no cenário atual de desenvolvimento e por oferecer a complexidade técnica ideal para testar nossas metodologias de LLMs.

O projeto está inserido no campo de Inteligência Artificial Generativa (GenAI), sendo um sistema que automatiza a criação de vídeos a partir de texto (text-to-video). Popularidade: Por ser um sistema de IA de código aberto altamente popular e ativo no GitHub, ele representa um código moderno que utiliza pipelines complexos e múltiplas integrações (APIs de voz, busca de vídeo, modelos de linguagem), o que o torna um alvo de estudo valioso.

ANÁLISE MANUAL DO CÓDIGO

A análise do README em inglês (que é a versão mais fácil de usar como evidência) sugere a combinação de três padrões arquiteturais principais: Model-View-Controller (MVC), Padrão de Serviço (Service Pattern) e Pipe-and-Filter (Pipeline).

Evidências:

- “Complete MVC architecture, clearly structured code, easy to maintain, supports both API and Web interface”. O padrão MVC é a estrutura primária do projeto, responsável por organizar a aplicação em camadas lógicas para manipulação de interface, lógica e dados.
- Services: O projeto realiza várias tarefas complexas que precisam ser isoladas em classes de serviço para manter o Controller "enxuto". Os arquivos de services dentro do projeto servem justamente para melhorar a arquitetura e isolar a lógica de negócio complexa
- O projeto descreve um fluxo sequencial e modular de processamento, onde a saída de uma etapa é a entrada da próxima (um pipe).

Sequência de Pipeline (Simplificada):

1. Entrada: Tópico ou Keyword.
2. Filtro 1: Geração do Roteiro (AI Copy).
3. Filtro 2: Busca e Seleção de Material de Vídeo (*stock footage*).
4. Filtro 3: Síntese de Voz e Geração de Legendas.
5. Filtro 4: Composição e *Render* do Vídeo Final.

Com essas evidências fica claro onde cada um desses padrões arquiteturais podem ser encontrados no código, tanto nas classes e nos arquivos quanto no próprio README.

MODELOS SELECIONADOS POR TASK

TASK	MODELO(S)
semantic search	sentence-transformers/all-MiniLM-L6-v2
text classification	google/flan-t5-xl e BART-Large-MNLI
code search	microsoft/graphcodebert-base mistralai/Mistral-7B-Instruct-v0.2 (auxiliar)

Text Classification

Uma análise automatizada foi realizada com o modelo Flan-T5-XL (para sumarização técnica) e BART-Large-MNLI (para classificação zero-shot), aplicados ao código fonte completo do repositório MoneyPrinterTurbo.

O modelo de linguagem escolhido foi o Flan-T5-XL (para sumarização técnica) e BART-Large-MNLI (para classificação zero-shot). O Flan-T5-XL foi selecionado devido sua flexibilidade em seguir prompts mais complexos, além de possuir um equilíbrio entre desempenho e performance se comparado a modelos maiores, também é um modelo open-source facilitando seu uso para pequenos projetos. O modelo BART-Large-MNLI BART combina um encoder (como BERT) e um decoder (semelhante ao GPT), o que dá a ele bastante flexibilidade de geração, permite usar a inferência textual para classificar textos, é uma boa escolha para sistemas de classificação onde você não tem muitos dados rotulados.

O repositório foi clonado e utilizando a linguagem de programação python, o código percorre recursivamente os diretórios do repositório e lê o conteúdo de arquivos com formatos .md, .py, .yaml, .yml, .toml e .txt e realiza uma limpeza de formatação do textos lidos. E assim utiliza um prompt para Flan-T5-XL resumir a seção do projeto focando em arquitetura e componentes. Com isso, o modelo BART-Large-MNLI classifica o texto usando a técnica zero-shot.

O processo foi dividido em duas etapas principais: geração do resumo técnico por blocos e posterior classificação arquitetural (interna e topológica).

Como reproduzir o experimento

Acesse o repositório através do link disponibilizado na primeira página deste documento.

Faça download do arquivo `colab_notebooks/text_classification.ipynb`.

Acesse o site [Google Colab](#).

Clique no botão “*Open Colab*”, localizado no canto superior direito.

Acesse a opção “*Upload*” no menu lateral à esquerda.

Clique na opção “*Browse*” e selecione o arquivo baixado no passo 2.

Clique no botão “*Run all*”, localizado no menu superior da tela.

Aguarde a execução do código, leva aproximadamente 12 minutos. O output será impresso na parte inferior da página.

Resultado Geral

Arquitetura interna predominante: Model-View-Controller (MVC)

Arquitetura de sistema (topológica): Client-Server

Essas classificações foram obtidas a partir do resumo técnico completo do projeto, processado em 30 blocos de texto ($\approx$ 234 mil caracteres no total).

Evidências técnicas extraídas

Durante a análise, o modelo Flan-T5-XL identificou padrões consistentes com uma estrutura em camadas separadas, com presença de controllers, serviços e interfaces de usuário (UI), tanto via Streamlit quanto FastAPI, o que reforça a presença do padrão MVC. A camada de controle orquestra a execução de serviços especializados e manipula a interação entre o front-end e o back-end.

No nível do sistema, a classificação Client-Server deriva da integração entre os componentes locais e APIs remotas, onde o projeto atua como cliente que consome serviços externos e oferece respostas processadas através de uma interface web.

Não há divisão explícita de microsserviços independentes, o que afasta a hipótese de uma arquitetura distribuída em microsserviços.

Tipo de Arquitetura	Classe Mais Provável	Score	Interpretação
Interna	MVC	0.2222	Estrutura organizada em controladores, serviços e camadas de visualização
Topológica	Client-Server	0.4236	O sistema funciona como um cliente que se comunica com servidores externos

Esses resultados convergem com a documentação textual do projeto, indicando que o MoneyPrinterTurbo utiliza um padrão híbrido, mas com núcleo fortemente baseado em MVC e uma implantação típica Client-Server, centralizando a execução em um único ambiente e se comunicando com serviços externos.

Variação entre execuções e consistência dos resultados

Durante a repetição dos testes, observou-se variação natural das classificações, em algumas execuções, o modelo classificou a arquitetura interna com Layered e a topologia como Distributed.

Essa flutuação é esperada e decorre de características inerentes aos modelos de linguagem:

- O texto do projeto é dividido em múltiplos blocos, e pequenas mudanças na divisão ou truncamento dos trechos podem alterar o foco semântico do resumo.
- A classificação zero-shot é probabilística e sensível a nuances de contexto; variações mínimas no resumo podem alterar ligeiramente a pontuação entre classes próximas.
- O código do projeto apresenta elementos comuns a diferentes estilos (camadas de serviço, controle e comunicação distribuída), o que torna mais de uma categoria plausível.

Mesmo com essa variação, as classificações mantiveram coerência estrutural, sempre orbitando em torno de padrões em camadas (MVC/Layered) e topologias centralizadas (Client-Server / Distributed), o que reforça a robustez da análise.

Conclusão

A LLM conseguiu identificar de forma coerente a estrutura e a organização do código. O projeto MoneyPrinterTurbo apresenta:

- Arquitetura Interna: Baseada em MVC, com serviços desacoplados que seguem boas práticas de separação de responsabilidades.
- Arquitetura Topológica: Client-Server, com comunicação via APIs externas e um fluxo centralizado.

Assim, o sistema pode ser descrito como uma aplicação monolítica organizada em MVC, operando dentro de um modelo Client-Server, o que combina simplicidade de manutenção com integração direta a provedores de IA. Adicionalmente, as evidências de 'serviços especializados' e 'fluxo centralizado' encontradas pelo LLM no resumo técnico são altamente compatíveis com o padrão *Pipe-and-Filter*, reforçando a análise manual.

Semantic Search

Uma análise automatizada da arquitetura do repositório escolhido pelo grupo foi conduzida utilizando um modelo de embedding em conjunto com técnicas de agrupamento não supervisionado (K-Means), o qual agrupa elementos com base em características semelhantes. O objetivo principal foi identificar os padrões arquiteturais, as camadas de responsabilidade e a organização modular do código.

O modelo de linguagem escolhido foi o sentence-transformers/all-MiniLM-L6-v2, priorizando a eficiência computacional e a acessibilidade. Este modelo é notável por ser extremamente leve e gerar resultados precisos com um processamento rápido, o que é ideal para ambientes com recursos limitados, como o Google Colab. Sua facilidade de uso o torna particularmente adequado para equipes que não possuem familiaridade avançada com LLMs, garantindo a geração eficaz de *embeddings* vetoriais de alta qualidade sem a necessidade de infraestrutura de hardware dedicada.

O repositório foi clonado e o código Python (.py), JSON/YAML (.json, .yaml) e arquivos de documentação (.md) foram extraídos como "snippets". Arquivos auxiliares e diretórios irrelevantes (venv, .git) foram ignorados para maximizar a relevância.

Cada snippet de código foi convertido em um vetor numérico de alta dimensão. A proximidade matemática entre esses vetores no espaço vetorial indica que os trechos de código têm propósitos ou lógicas semelhantes.

Para utilização do K-Means foi necessário encontrar um número k ideal de grupos, essa escolha foi baseada no Método do Cotovelo (*Elbow Method*), o qual testa vários valores para k , e após plotar um gráfico é feita análise em busca do ponto de estabilidade, que neste caso foi $K = 8$. Os arquivos foram automaticamente distribuídos em 8 clusters, onde cada cluster representa um conjunto de responsabilidades coesas.

Para transformar o grupo de arquivos (cluster) em um padrão identificável, foi utilizada uma heurística que calculou a distância de cada um dos arquivos no cluster até o centro do cluster para selecionar os 5 arquivos mais ao centro, e o arquivo mais ao centro dentre esses 5 arquivos é usado para coletar palavras-chave que identificam o cluster.

Como reproduzir o experimento

Acesse o repositório através do link disponibilizado na primeira página deste documento.

Faça download do arquivo colab_notebooks/semantic_search.ipynb.

Acesse o site [Google Colab](#).

Clique no botão "Open Colab", localizado no canto superior direito.

Acesse a opção "Upload" no menu lateral à esquerda.

Clique na opção "Browse" e selecione o arquivo baixado no passo 2.

Clique no botão "Run all", localizado no menu superior da tela.

Aguarde a execução do código, leva aproximadamente 1 minuto. O output será impresso na parte inferior da página.

Resultado Geral

Padrão Predominante	Arquitetural	Camadas Identificadas
Layered (Arquitetura em Camadas)	Architecture	Apresentação (Controller), Lógica de Negócio (Service), Domínio (Model/Schema).

A análise de *clustering* demonstrou que o repositório MoneyPrinterTurbo emprega claramente o **Padrão Arquitetural em Camadas** (*Layered Architecture*). O Layered Architecture encontrado é a base para o padrão **Model-View-Controller (MVC)**, previamente identificado. Os clusters revelam as camadas de Apresentação (View/Controller), Lógica (Service) e Dados (Model).

O agrupamento dos códigos em **clusters semanticamente coerentes** revela três camadas principais e bem definidas:

1. **Apresentação/Controle (Cluster 3):** Endpoints de API e roteamento (controllers, router).
2. **Lógica de Aplicação (Clusters 0 & 5):** Os serviços (services) que contêm a inteligência do negócio (geração de conteúdo, manipulação de mídia).
3. **Domínio/Modelos (Cluster 6):** Onde os dados e as estruturas de comunicação (models/schema) são definidos.

Uma notável inconsistência foi observada no **Cluster 7**, onde o agrupamento isolou arquivos de configuração de infraestrutura (.github/*.yaml). Embora a coesão do cluster seja alta (os arquivos são muito parecidos entre si), a sugestão automática de **'Repository/DAO'** é factualmente incorreta. Esta falha de classificação ocorre porque o vetor do *embedding* do arquivo .yaml é semanticamente muito distante do código, causando um 'salto' incorreto na heurística.

Conclusão

O mapeamento da LLM, mesmo com a inconsistência do cluster 7, conseguiu de forma satisfatória identificar um padrão arquitetural através das técnicas de *clustering* e *semantic search*, validando que a estrutura é clara e segue as boas práticas de modularização. A identificação do Layered Architecture, segmentado em camadas de Apresentação/Controle, Lógica de Aplicação (Services) e Domínio, confirma tanto a Análise Manual quanto o resultado da Text Classification.

Code Search

Foi conduzida uma análise automatizada de **detecção de padrões arquiteturais e de projeto** utilizando o modelo **GraphCodeBERT**, especializado em compreensão semântica de código-fonte. O objetivo do experimento foi avaliar a capacidade do modelo de identificar **estruturas e padrões arquiteturais** recorrentes em um repositório Python real — o projeto **MoneyPrinterTurbo** — através de uma abordagem de **code search semântico supervisionado por embeddings**.

O modelo GraphCodeBERT-base foi escolhido devido à sua capacidade de entender as relações sintáticas entre os tokens gerados com o uso de grafos. O uso da Mistral-7B-Instruct-v0.2 como auxiliar se deu principalmente por ser um modelo leve e conversacional, resumindo assim a análise feita pelo GraphCodeBERT-base.

O pipeline foi dividido em duas etapas principais:

1. **Extração semântica e classificação de padrões de projeto** com o modelo GraphCodeBERT.
2. **Análise interpretativa automática** via LLM (Mistral-7B-Instruct), responsável por

sintetizar os resultados e explicar as características arquiteturais predominantes.

Como reproduzir o experimento

Acesse o repositório através do link disponibilizado na primeira página deste documento.

Faça download do arquivo `colab_notebooks/code_search.ipynb`.

Acesse o site [Google Colab](https://colab.research.google.com/).

Clique no botão “*Open Colab*”, localizado no canto superior direito.

Acesse a opção “*Upload*” no menu lateral à esquerda.

Clique na opção “*Browse*” e selecione o arquivo baixado no passo 2.

Clique no botão “*Run all*”, localizado no menu superior da tela.

Aguarde a execução do código, leva aproximadamente 9 minutos. O output será impresso na parte inferior da página.

Etapa 1 — Code Search com GraphCodeBERT

O modelo GraphCodeBERT-base, desenvolvido pela Microsoft, foi utilizado para gerar embeddings vetoriais de cada arquivo .py no repositório, permitindo medir a similaridade semântica entre o código e descrições textuais de padrões de projeto clássicos (como Singleton, Factory Method, Adapter, MVC e Clean Architecture).

Esses embeddings foram comparados usando a métrica de similaridade cosine similarity, e o arquivo foi rotulado com o padrão mais semelhante, considerando um limiar de 0.65.

O processo analisou automaticamente 34 arquivos do repositório MoneyPrinterTurbo.

Resultado Geral

Padrões de Projeto Predominantes:

- Clean Architecture
- Singleton

Tipo de Padrão	Frequência	Interpretação
Clean Architecture	14 arquivos	Indica uma separação clara entre camadas de domínio, serviços e interface.
Singleton	8 arquivos	Indica presença de gerenciadores e controladores únicos centralizados.
Nenhum	9 arquivos	Códigos auxiliares, testes e módulos genéricos sem estrutura reconhecível.

Evidências Técnicas Extraídas

Durante a análise, o GraphCodeBERT identificou **ocorrências recorrentes do padrão Singleton** em arquivos como `config.py`, `memory_manager.py`, `redis_manager.py` e `main.py`. Esses módulos desempenham papéis de controle global e gerenciamento de estado — características típicas do Singleton.

Por outro lado, os módulos como `router.py`, `video.py`, `llm.py` e `schema.py` apresentaram alta similaridade com **Clean Architecture**, destacando uma estrutura em **camadas bem definidas**, que separa claramente lógica de negócio, domínio e infraestrutura.

Essa organização modular reflete um **núcleo monolítico estruturado em Clean Architecture**, com instâncias únicas (Singletons) responsáveis pela coordenação de serviços e configurações — um arranjo híbrido frequente em sistemas de IA com pipeline centralizado.

Etapa 2 — Interpretação Semântica com LLM

Após a detecção automática, os resultados foram analisados pelo modelo **Mistral-7B-Instruct**, que interpretou os padrões encontrados e produziu um resumo técnico.

A LLM confirmou a coexistência dos padrões **Clean Architecture** e **Singleton** como fundamentais para a estrutura do projeto, destacando que o primeiro organiza as camadas de domínio e lógica, enquanto o segundo gerencia recursos compartilhados e pontos únicos de acesso.

Essa combinação sugere uma **arquitetura monolítica modularizada**, otimizada para processamento centralizado e comunicação via APIs — com princípios de **Clean Architecture** aplicados sobre uma base **Client-Server**.

Conclusão

A abordagem de **Code Search baseada em embeddings** com GraphCodeBERT demonstrou ser eficaz para identificação automática de padrões de projeto e características arquiteturais em código-fonte.

O projeto **MoneyPrinterTurbo** apresenta:

- **Arquitetura Interna:** Baseada em **Clean Architecture**, com camadas bem definidas e forte separação de responsabilidades.
- **Padrões Reforçadores:** **Singletons** implementados para controle de configuração, memória e recursos compartilhados.
- **Arquitetura Topológica:** **Client-Server**, com execução centralizada e comunicação com serviços externos.

Essas evidências indicam uma **estrutura híbrida**, onde o Clean Architecture fornece modularidade e o Singleton garante coerência e controle global, resultando em uma aplicação organizada, extensível e de manutenção simplificada.

A análise semântica de código, embora eficaz na identificação de padrões, apresentou duas limitações metodológicas que afetam a validade das conclusões. Primeiramente, os modelos Encoder utilizados (como o GraphCodeBERT) focam primariamente na coesão estrutural intra-arquivo, logo, a metodologia demonstra dificuldade em identificar padrões arquiteturais que dependem da orquestração e relações entre arquivos distintos (como Injeção de Dependência).

Em segundo lugar, o Code Search baseia sua detecção em um conjunto pré-determinado e fechado de padrões canônicos; isso significa que, caso o projeto MoneyPrinterTurbo utilize algum padrão ou variação arquitetural não listada em nosso script de comparação, ele não será detectado ou será classificado incorretamente.

Análise geral dos resultados

Metodologia	Modelo(s) Utilizado(s)	Método de Análise	Unidade de Análise	Grau de Detalhe	Resultado Chave Confirmado	Tempo de execução (min)
Text Classification	Flan-T5-XL & BART-L	Dedutiva (Interpretação de Resumo)	Repositório Completo (Texto)	Baixo	MVC / Client-Server	11:52
Semantic Search (Clustering)	all-MiniLM-L6-v2	Indutiva (Agrupamento Vetorial)	Arquivo (Código)	Médio	Existência física das camadas (Service, Controller).	01:46
Code Search	GraphCodeBERT & Mistral-7B	Semântica (Similaridade de Código)	Arquivo (Código)	Alto	Padrões granulares (Singleton) e estrutura (Clean Architecture).	09:12

Avaliação da Efetividade dos Modelos

Os modelos Semantic Search (Clustering) e Code Search foram os mais efetivos para a atividade, pois fornecem evidências técnicas diretas extraídas do código-fonte, superando a classificação baseada em texto.

Justificativa:

1. **Grau de Detalhe e Especificidade (Code Search):** O GraphCodeBERT é superior por ser pré-treinado em código, o que lhe permite identificar padrões complexos como Singleton e Clean Architecture através da análise semântica direta, fornecendo contagens precisas (8 Singletons e 14 Clean Architecture). Este detalhe é fundamental para validar o design interno do software.
2. **Validação Estrutural (Semantic Search):** O método de Clustering atua como uma prova indutiva da arquitetura. Ele comprovou que a estrutura Layered Architecture existe fisicamente ao agrupar os arquivos em camadas coesas (Service e Controller), validando o MVC de forma empírica.
3. **Limitação (Text Classification):** O método Text Classification, embora consistente no resultado MVC, apresentou o score de confiança mais baixo (0.2222). Sua dependência da interpretação de texto o torna menos robusto para detalhamento arquitetural, que exige a análise da sintaxe e semântica do código.

A inclusão do tempo de execução reforça a avaliação de efetividade: o Semantic Search (Clustering) foi o mais rápido (01:46 min) e demonstrou alta eficácia na validação estrutural (Arquitetura em camadas), sendo a solução mais eficiente. Por outro lado, o Code Search levou 09:12 min, um trade-off aceitável entre tempo e granularidade alta. Já o Text Classification, além de ter o menor score de confiança, consumiu o maior tempo (11:52 min), indicando baixa eficiência para esta tarefa.

Conclusão: A combinação do **Clustering (prova estrutural)** com o **Code Search (prova de padrões)** forneceu o panorama mais completo e validado da arquitetura do projeto.

RELAÇÃO DOS PARTICIPANTES

NOME	MATRÍCULA	CONTRIBUIÇÃO
Fernanda da Silva Santos	202300061581	Escolha do projeto alvo de análise, uso da LLM <i>sentence-transformers/all-MiniLM-L6-v2</i> para análise do código-fonte, escrita do relatório, gravação do vídeo
Gustavo Rodrigues dos Santos	202300061939	Escolha do projeto alvo de análise, uso da LLM FLAN-T5-XL para análise do código fonte do repositório e a LLM BART-Large-MNLI para classificar o padrão da arquitetura, escrita e revisão do relatório, gravação do vídeo.
Joao Santos Rocha	202300061948	Escolha do projeto alvo de análise,

		uso das LLMs GraphCodeBERT para detecção de padrões de código e Mistral-7B-Instruct para análise semântica, escrita do relatório, gravação do vídeo.
Jose Henrique Souza Santana	202300061699	Escolha do projeto alvo de análise, uso das LLMs GraphCodeBERT para detecção de padrões de código e Mistral-7B-Instruct para análise semântica, escrita do relatório, gravação do vídeo.
Jose Luiz de Jesus Souza	201800138910	O integrante não contribuiu com a atividade.
Lais Santos de Sousa	202100115364	Escolha do projeto alvo de análise, uso da LLM sentence-transformers/all-MiniLM-L6-v2 por meio de script para análise do código fonte, escrita do relatório, gravação do vídeo.
Rafael de Jesus Santos	202300114489	Escolha do projeto alvo de análise, uso da LLM FLAN-T5-XL para gerar um resumo técnico do projeto e a LLM BART-Large-MNLI para classificar o padrão arquitetural, escrita do relatório e gravação do vídeo.
Tasso Marcel de Oliveira	202300061984	Escolha do projeto alvo de análise, identificação manual dos padrões arquitetuais, uso da LLM FLAN-T5-XL para gerar um resumo técnico do projeto e a LLM BART-Large-MNLI para classificar o padrão arquitetural, escrita do relatório e gravação do vídeo.